



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



KHOA TOÁN - TIN
Faculty of Mathematics and Informatics

Triển khai hệ thống Từ điển Anh - Việt

Nhóm 777 chủ đề 7

Giảng viên : Vũ Thành Nam
Mã lớp : 169313
Sinh viên : Phạm Xuân Việt 202419016
Lã Mạnh Cường 202418860

Contents

LỜI MỞ ĐẦU	5
1 TỔNG QUAN ĐỀ TÀI	6
1.1 Mô tả đề tài	6
1.2 Yêu cầu chức năng	6
1.3 Yêu cầu phi chức năng	6
2 PHÂN TÍCH YÊU CẦU	8
2.1 Phân tích chức năng theo nhóm trách nhiệm	8
2.2 Nhận xét và Định hướng thiết kế kiến trúc	9
3 TRIỂN KHAI CHƯƠNG TRÌNH	10
3.1 Cấu trúc thư mục	10
3.2 Mô tả chi tiết các lớp	11
3.2.1 Lớp template <code>DynamicArray<T></code>	11
3.2.2 Lớp <code>Word</code>	11
3.2.3 Strategy Pattern: <code>ISearchStrategy</code>	11
3.2.4 Lớp <code>Dictionary</code>	12
3.2.5 Lớp <code>History</code> (Linked List + Rule of Three)	12
3.2.6 Facade: <code>Application</code>	12
3.3 Định dạng file dữ liệu	13
4 THIẾT KẾ HƯỚNG ĐỐI TƯỢNG	14
4.1 Hệ thống lớp – sơ đồ tổng quan	14
4.2 Các nguyên tắc OOP đã áp dụng	15
4.2.1 Đóng gói (Encapsulation)	15
4.2.2 Kế thừa (Inheritance)	15
4.2.3 Đa hình (Polymorphism)	15
4.2.4 Trừu tượng hoá (Abstraction)	16
4.3 Các mẫu thiết kế (Design Patterns)	16
4.3.1 Strategy Pattern – Chiến lược tìm kiếm	16
4.3.2 Facade Pattern – Lớp <code>Application</code>	16

4.3.3	Template Method ngầm – IPersistable	17
4.4	Hệ thống Exception	17
5	KIỂM THỬ	18
5.1	Kỹ thuật kiểm thử	18
5.2	Demo	20
6	TỔNG KẾT CÁC KỸ THUẬT ĐÃ VẬN DỤNG	23
6.1	Kỹ thuật về thiết kế hướng đối tượng (OOP)	23
6.2	Các mẫu thiết kế (Design Patterns)	23
6.3	Kỹ thuật về thuật toán và cấu trúc dữ liệu	24
A	MÃ NGUỒN CÁC THÀNH PHẦN CHÍNH	25
A.1	Hàm main	25
A.2	Facade: Application	26
A.2.1	Application.h	26
A.2.2	Application.cpp	27
A.3	Hệ thống Exception	28
A.4	Interface IPersistable	30
A.5	Strategy Pattern – ISearchStrategy	31
A.6	Dictionary.cpp (trích)	32
A.7	History.cpp (Rule of Three)	34

PHÂN CÔNG NHIỆM VỤ

Sinh viên thực hiện	Nhiệm vụ	KQ
Phạm Xuân Việt	- Thiết kế kiến trúc hướng đối tượng (OOP), xây dựng hệ thống xử lý nghiệp vụ lỗi và các cấu trúc dữ liệu tự tạo (mảng động <code>DynamicArray</code>, <code>LinkedList</code> cho <code>History</code>). - Xây dựng hệ thống xử lý ngoại lệ (<code>Exception Hierarchy</code>) - Áp dụng Strategy Pattern để đóng gói thuật toán tìm kiếm chính xác (<code>ExactSearch</code>) và tìm kiếm gần đúng (<code>ApproximateSearch</code>).	Đạt 100%
Lã Mạnh Cường	- Xây dựng giao diện console (<code>ConsoleUI</code>). - Áp dụng Facade Pattern (thông qua lớp <code>Application</code>) nhằm cung cấp giao diện thống nhất để khởi tạo, điều phối hệ thống con và quản lý vòng đời ứng dụng. - Thiết kế cơ chế lưu/tải dữ liệu file qua giao diện <code>IPersistable</code> - Viết kịch bản kiểm thử và soạn báo cáo kỹ thuật.	Đạt 100%
Thực hiện chung	- Tích hợp các module hệ thống, tiến hành kiểm tra chéo mã nguồn và nghiệm thu luồng thực thi tổng thể.	Đạt 100%

LỜI MỞ ĐẦU

Project "Từ điển Anh - Việt".

Hệ thống được phát triển với trọng tâm vào việc tự chủ quản trị tài nguyên bộ nhớ (không phụ thuộc vào các thư viện cấu trúc tiêu chuẩn), đồng thời phân tách khắt khe các tầng logic nghiệp vụ thông qua các mẫu thiết kế (Design Patterns) như Strategy và Facade.

Kết quả đạt được là một kiến trúc phần mềm có tính toàn vẹn cao, dễ bảo trì và mở rộng.

Nhóm em xin gửi lời cảm ơn sâu sắc đến Giảng viên môn học Vũ Thành Nam. Những kiến thức trên lớp, phương pháp tư duy logic và các phản biện kỹ thuật từ thầy đã giúp nhóm em có thể giải quyết các vấn đề và hoàn thiện hệ thống phần mềm này.

Trong dự án này, nhóm có ứng dụng sự hỗ trợ của AI. Cụ thể như sau:

- **Công đoạn 1: Nghiên cứu lý thuyết và Thiết kế kiến trúc**

- **Nhóm chủ động:** Nhận diện bài toán, thiết lập các ràng buộc phi chức năng (từ chối sử dụng `std::vector`, `std::list`) và trực tiếp ra quyết định lựa chọn kiến trúc hệ thống (áp dụng Strategy Pattern, Facade Pattern).

- **AI hỗ trợ:** Đóng vai trò phân tích lý thuyết lập trình hướng đối tượng (OOP) và tạo các ví dụ minh họa cho hệ thống mẫu thiết kế (Design Patterns).

- **Công đoạn 2: Đặc tả tiến trình và Triển khai mã nguồn**

- **Nhóm chủ động:** Liệt kê chi tiết các yêu cầu chức năng, vẽ lưu đồ tiến trình (workflow) và định nghĩa các hợp đồng giao diện cốt lõi (IPersistable, ISearchStrategy). Nhóm trực tiếp can thiệp, điều chỉnh và tái cấu trúc (refactor) toàn bộ mã do AI sinh ra để ép tuân thủ Quy tắc số 3 (Rule of Three) và nguyên lý quản trị vùng nhớ RAII.

- **AI hỗ trợ:** Tiếp nhận các đặc tả giao diện từ nhóm để sinh ra các đoạn mã khung, khởi tạo cấu trúc tệp tin tĩnh và hỗ trợ viết các hàm tiện ích lặp đi lặp lại nhằm tối ưu hóa thời gian lập trình.

- **Công đoạn 3: Kiểm thử, Gỡ lỗi và Tối ưu hóa hệ thống**

- **Nhóm chủ động:** Trực tiếp thiết kế kịch bản kiểm thử, đưa ra các trường hợp biên cực đoan (chuỗi rỗng, tra cứu vượt giới hạn, xung đột tệp tin). Đánh giá độ an toàn và quyết định áp dụng hay loại bỏ các bản vá lỗi mà AI đề xuất.

- **AI hỗ trợ:** Quét và giải thích các thông điệp lỗi từ trình biên dịch (compiler/linker logs), phân tích các điểm nghẽn trong tệp cấu hình Makefile và đề xuất các phương án gỡ lỗi để nhóm tham chiếu.

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

1.1. Mô tả đề tài

Đề tài Chủ đề 7: Từ điển Anh – Việt.

Mục tiêu xây dựng chương trình tra cứu từ vựng với các tính năng cốt lõi của một từ điển điện tử: quản lý từ vựng, tra cứu chính xác, tra cứu gần đúng dựa trên thuật toán so khớp chuỗi, lưu lịch sử và đánh dấu từ yêu thích.

Phiên bản trình bày trong báo cáo này là phiên bản nâng cấp theo hướng đối tượng (OOP). Việc nâng cấp nhằm minh họa rõ nét các nguyên tắc và mẫu thiết kế hướng đối tượng đã học, đồng thời cải thiện độ mở rộng và bảo trì của hệ thống.

1.2. Yêu cầu chức năng

- Quản lý từ vựng: thêm, sửa, xóa từ. Mỗi từ bao gồm nghĩa tiếng Việt, ví dụ, từ đồng nghĩa.
- Tra cứu chính xác (không phân biệt hoa - thường).
- Tra cứu gần đúng dựa trên khoảng cách Levenshtein, gợi ý các từ tương tự khi người dùng gõ sai.
- Lịch sử tra cứu (most-recently-used, có giới hạn).
- Đánh dấu từ vựng yêu thích.
- Lưu trữ dữ liệu trong file text.

1.3. Yêu cầu phi chức năng

Đề bài yêu cầu không sử dụng các cấu trúc dữ liệu nâng cao hay thư viện có sẵn (`std::vector`, `std::list`, `std::map`, `std::sort`, ...). Ngoài ra, phiên bản OOP còn đặt thêm các yêu cầu nội bộ:

- Áp dụng đầy đủ 4 trụ cột OOP: đóng gói, kế thừa, đa hình, trừu tượng hoá.
- Áp dụng ít nhất 2 mẫu thiết kế (design pattern).
- Tách biệt rõ tầng dữ liệu, tầng thuật toán, tầng giao diện.

- Sử dụng exception thay cho mã trả về bool/int để báo lỗi.

CHƯƠNG 2

PHÂN TÍCH YÊU CẦU

2.1. Phân tích chức năng theo nhóm trách nhiệm

Trên cơ sở các yêu cầu chức năng, hệ thống được phân rã thành các nhóm trách nhiệm. Mỗi nhóm sau này sẽ tương ứng với một (hoặc một số) lớp:

Nhóm trách nhiệm	Mô tả
Mô hình dữ liệu	Biểu diễn 1 từ vựng (Word) với các trường: từ tiếng Anh, danh sách nghĩa, ví dụ, đồng nghĩa.
Quản lý kho từ	Tập hợp các từ (Dictionary), hỗ trợ thêm/sửa/xóa và truy vấn.
Thuật toán tìm kiếm	Các chiến lược khác nhau: chính xác, gần đúng. Phải có khả năng đổi chiến lược runtime và dễ mở rộng.
Trạng thái phụ trợ	Lịch sử tra cứu (FIFO có giới hạn), danh sách yêu thích.
Bền vững hoá	Lưu/tải dữ liệu xuống/từ file text. Áp dụng cho Dictionary, History, Favorites – cùng một giao diện.
Giao diện người dùng	Hiển thị menu, nhận lệnh, hiển thị kết quả. Phải tách biệt với phần logic để có thể thay thế bằng GUI sau này.
Điều phối	Khởi tạo, kết nối các thành phần, quản lý vòng đời.

2.2. Nhận xét và Định hướng thiết kế kiến trúc

Phân tích yêu cầu kỹ thuật cốt lõi, từ đó hệ thống hóa các nhóm giải pháp tương ứng:

Thách thức / Quan sát	Định hướng kỹ thuật kiến trúc
1. Đồng nhất hoá luồng I/O (Persistence)	Dictionary, History, Favorites có cấu trúc nội bộ khác biệt nhưng đều cần đọc/ghi dữ liệu. Trừu tượng hóa qua interface IPersistable để xử lý lưu trữ đa hình, độc lập với cấu trúc bộ nhớ vật lý bên dưới.
2. Quản lý sự thay đổi thuật toán (Strategy)	Gói các chiến lược tra cứu (chính xác, xấp xỉ Levenshtein) vào các lớp độc lập kế thừa chung interface ISearchStrategy. Đảm bảo nguyên lý Open/Closed, cho phép thay đổi thuật toán tại runtime mà không can thiệp logic lõi.
3. Cô lập phân hệ giao diện (UI Decoupling)	Tách biệt giao diện khỏi logic nghiệp vụ thông qua interface IUserInterface. Che giấu chi tiết tương tác dòng lệnh (ConsoleUI), đảm bảo tính mở để nâng cấp lên giao diện đồ họa (GuiUI) trong tương lai.
4. Quản trị tài nguyên thô và bộ nhớ (RAII)	Chấp nhận ràng buộc cấm sử dụng cấu trúc chuẩn (std::vector, list). Tự thiết kế vùng nhớ cho DynamicArray<T> và áp dụng nghiêm ngặt Quy tắc số 3 (Rule of Three), RAII nhằm triệt tiêu rò rỉ bộ nhớ.
5. Khử liên kết chặt (Facade Pattern)	Ngăn chặn tình trạng phụ thuộc chéo giữa các module. Dùng lớp Application làm mặt tiền điều phối (Facade) trực tiếp quản lý vòng đời và tiêm phụ thuộc (Dependency Injection), giúp hàm main tối giản.
6. Kiểm soát trạng thái lỗi (Exceptions)	Loại bỏ mã lỗi tĩnh (return codes) để bảo toàn luồng hướng đối tượng. Xây dựng cây ngoại lệ kế thừa từ DictionaryException nhằm cô lập luồng xử lý nghiệp vụ khỏi luồng bắt lỗi.

CHƯƠNG 3

TRIỂN KHAI CHƯƠNG TRÌNH

3.1. Cấu trúc thư mục

TuDienAnhViet/	
-- src/	
-- app/	Miền Điều phối (Facade & Bootstrap)
-- Application.h / .cpp	Facade Pattern - điều phối hệ thống
-- main.cpp	Entry point: Khởi tạo Application và chạy
-- core/	Miền Cốt lõi (Cấu trúc nền tảng)
-- DynamicArray.h	Template mảng động (generic)
-- Word.h / .cpp	Class thực thể: encapsulation đầy đủ, operator overloading
-- persistence/	Miền Bền vững hóa (Bảo toàn trạng thái)
-- IPersistable.h	Interface save/load
-- Dictionary.h / .cpp	Implement IPersistable (Quản lý tập Word)
-- Favorites.h / .cpp	Implement IPersistable
-- History.h / .cpp	Implement IPersistable
	(Linked list, Rule of Three)
-- search/	Miền Thuật toán (Strategy Pattern)
-- ISearchStrategy.h	Interface chiến lược tìm kiếm
-- ExactSearch.h / .cp	Chiến lược: Khớp chuỗi tuyệt đối
-- ApproximateSearch.h / .cpp	Khớp chuỗi bằng Khoảng cách Levenshtein
-- ui/	Miền Giao diện
-- IUserInterface.h	Interface tương tác
-- ConsoleUI.h / .cpp	Implement IUserInterface
-- utils/	Miền Tiện ích & Biệt lệ
-- Exceptions.h	Mở rộng từ std::exception
-- StringUtils.h / .cpp	Xử lý chuỗi (Levenshtein, toLower, trim, split)

```
|
|-- data/
|   |-- dictionary.txt
|   |-- history.txt
|   \-- favorites.txt
|
|-- docs/
|   \-- class_diagram.png
|
|-- Makefile
|-- build.bat
\-- README.md
```

3.2. Mô tả chi tiết các lớp

3.2.1. Lớp template *DynamicArray<T>*

Cài đặt mảng động kiểu generic, đầy đủ Rule of Three (copy constructor, copy assignment, destructor). Cơ chế nhân đôi capacity khi đầy cho push_back đạt $O(1)$ amortized.

3.2.2. Lớp *Word*

Đại diện một từ với 4 thuộc tính private. Cài đặt operator==, operator!= (so sánh theo english không phân biệt hoa – thường) và friend operator« (in ra ostream). Việc dùng operator« thay cho hàm print() có ưu điểm: cú pháp tự nhiên, có thể ghép nối (std::cout « word1 « word2), tương thích với mọi std::ostream (cả console lẫn file).

3.2.3. Strategy Pattern: *ISearchStrategy*

Đoạn code thể hiện rõ Strategy Pattern trong ConsoleUI::handleLookup:

```
1 // Bước 1: dùng Strategy ExactSearch
2 DynamicArray<SearchResult> exactResults;
3 dict_.search(exactSearch_, q, exactResults);
4
5 if (!exactResults.empty()) {
6     const Word& w = dict_.at(exactResults[0].wordIndex);
7     std::cout << w;
8     history_.add(w.getEnglish());
9     return;
10 }
```

```

11
12 // Bước 2: chuyển sang Strategy ApproximateSearch
13 DynamicArray<SearchResult> approxResults;
14 dict_.search(approxSearch_, q, approxResults);

```

Cùng một phương thức `dict_.search()` được gọi với hai strategy khác nhau và cho hai hành vi khác nhau – đây chính là điểm mạnh của đa hình.

3.2.4. Lớp Dictionary

Quản lý `DynamicArray<Word>`. Cung cấp các phép CRUD ném exception khi gặp lỗi:

```

1 void Dictionary::addWord(const Word& w) {
2     if (findIndexExact(w.getEnglish()) != -1) {
3         throw DuplicateWordException(w.getEnglish());
4     }
5     words_.push_back(w);
6 }
7
8 void Dictionary::deleteWord(const std::string& english) {
9     int idx = findIndexExact(english);
10    if (idx == -1) {
11        throw WordNotFoundException(english);
12    }
13    words_.removeAt(idx);
14 }

```

3.2.5. Lớp History (Linked List + Rule of Three)

History dùng danh sách liên kết đơn, quản lý heap thông qua `new/delete`. Vì có pointer member nên phải cài đặt đầy đủ Rule of Three: copy constructor, copy assignment, destructor. Hàm `copyFrom()` thực hiện deep copy theo đúng thứ tự để giữ nguyên ngữ nghĩa “most-recently-used”.

3.2.6. Facade: Application

```

1 void Application::initialize() {
2     try {
3         dict_.loadFromFile(DICT_FILE);
4     } catch (const FileIOException&) {
5         // OK: lan dau chay, chua co file
6     }

```

```
7     try { history_.loadFromFile(HIST_FILE); }
8     catch (const FileIOException&) {}
9     try { favorites_.loadFromFile(FAV_FILE); }
10    catch (const FileIOException&) {}
11
12    // Polymorphism: có thể thay bằng GuiUI sau này
13    ui_ = new ConsoleUI(dict_, history_, favorites_);
14 }
```

3.3. Định dạng file dữ liệu

File dictionary.txt giữ nguyên định dạng của phiên bản trước để tương thích ngược:

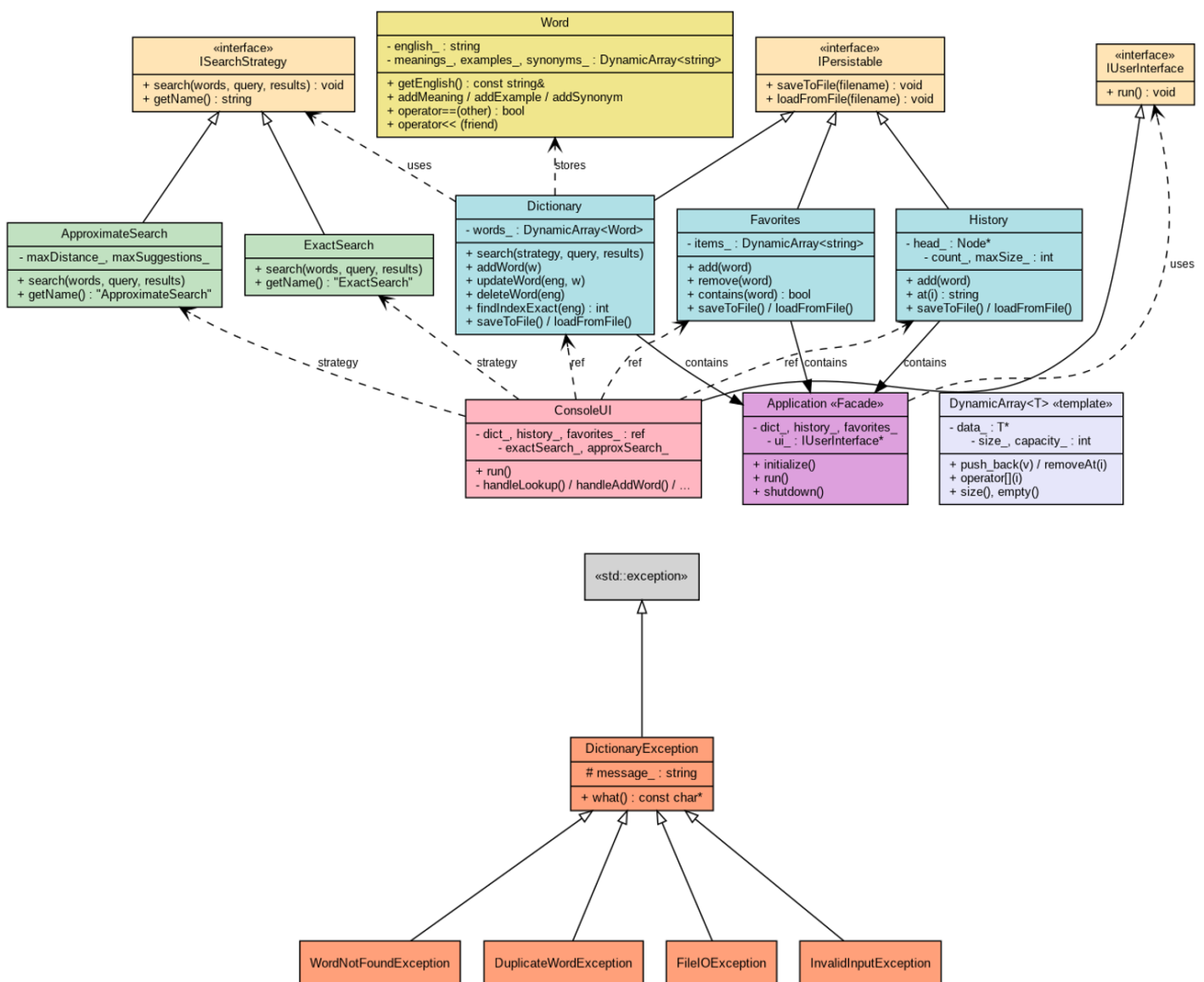
```
@english=hello
meaning=xin chào|chào hỏi
example=Hello, how are you?|Say hello to your mom.
synonym=hi|hey|greetings
@end
```

history.txt và favorites.txt mỗi dòng một từ tiếng Anh.

CHƯƠNG 4

4.1. Hệ thống lớp – sơ đồ tổng quan

Sơ đồ lớp dưới đây thể hiện toàn bộ kiến trúc của hệ thống. Các ô màu vàng nhạt là interface, các ô khác là các lớp cụ thể, các mũi tên rộng là quan hệ kế thừa, các mũi tên đứt nét là quan hệ phụ thuộc.



4.2. Các nguyên tắc OOP đã áp dụng

4.2.1. Đóng gói (Encapsulation)

Tất cả các lớp đều có trường dữ liệu private, được truy cập thông qua getter/setter công khai.

Ví dụ lớp Word:

```

1  class Word {
2  private:
3      std::string english_;
4      DynamicArray<std::string> meanings_;
5      DynamicArray<std::string> examples_;
6      DynamicArray<std::string> synonyms_;
7
8  public:
9      const std::string& getEnglish() const { return english_; }
10     void addMeaning(const std::string&m) {
11         meanings_.push_back(m); }
12     // ...
13 };

```

Lưu ý quy ước đặt tên: trường private có hậu tố underscore (english_, meanings_) – một quy ước phổ biến trong các dự án C++ chuyên nghiệp (Google Style Guide).

4.2.2. Kế thừa (Inheritance)

Có ba cây kế thừa chính trong hệ thống:

- IPersistable (interface) ← Dictionary, History, Favorites
- ISearchStrategy (interface) ← ExactSearch, ApproximateSearch
- IUIInterface (interface) ← ConsoleUI
- std::exception ← DictionaryException ← WordNotFoundException, DuplicateWordException
FileIOException, InvalidInputException

4.2.3. Đa hình (Polymorphism)

Đa hình động được sử dụng tại nhiều điểm:

- Dictionary::search nhận một const ISearchStrategy& – cùng một lời gọi – (strategy, query, results) có thể thực thi thuật toán khác nhau tùy vào kiểu thực thể của strategy.

- Application giữ con trỏ `IUserInterface*` – sau này thay `ConsoleUI` bằng `GuiUI` chỉ cần đổi một dòng `new ConsoleUI(...)` thành `new GuiUI(...)`.
- Catch `DictionaryException` bắt được tất cả exception con nhờ đa hình của hàm `what()`.

4.2.4. Trừu tượng hoá (Abstraction)

Các interface (`IPersistable`, `ISearchStrategy`, `IUserInterface`) đều là abstract class chứa pure virtual function. Client code không cần biết chi tiết cài đặt – chỉ làm việc thông qua interface.

4.3. Các mẫu thiết kế (Design Patterns)

4.3.1. Strategy Pattern – Chiến lược tìm kiếm

Strategy Pattern đóng gói các thuật toán khác nhau vào các lớp riêng và cho phép thay đổi thuật toán tại thời điểm chạy. Trong hệ thống này:

- Interface `ISearchStrategy` định nghĩa hợp đồng `search(words, query, results)`.
- `ExactSearch` cài đặt thuật toán tìm chính xác (case-insensitive).
- `ApproximateSearch` cài đặt thuật toán tìm gần đúng (Levenshtein).
- `Dictionary::search` nhận một `const ISearchStrategy&` – không phụ thuộc vào thuật toán cụ thể.

Lợi ích thực tế: trong tác vụ tra cứu, `ConsoleUI` thử `ExactSearch` trước, nếu không có kết quả thì chuyển sang `ApproximateSearch` – đổi chiến lược động ngay trong một lần thao tác. Khi muốn thêm thuật toán mới (`PrefixSearch` dùng Trie, `PhoneticSearch` dùng Soundex, ...) chỉ cần tạo lớp mới kế thừa `ISearchStrategy` mà không sửa `Dictionary`.

4.3.2. Facade Pattern – Lớp Application

Facade Pattern cung cấp một giao diện thống nhất cho một tập các giao diện trong một hệ thống con. Application gói tất cả các thành phần (`Dictionary`, `History`, `Favorites`, `IUserInterface`) và expose ba phương thức đơn giản: `initialize()`, `run()`, `shutdown()`.

Nhờ vậy `main()` chỉ còn khoảng 15 dòng:

```
1 int main() {
2     try {
3         Application app;
4         app.initialize();
```

```

5         app.run();
6         app.shutdown();
7     } catch (const DictionaryException& e) {
8         std::cerr << "Loi: " << e.what() << "\n";
9         return 1;
10    }
11    return 0;
12 }

```

4.3.3. Template Method ngầm – IPersistable

IPersistable không phải Template Method điển hình, nhưng chia sẻ tinh thần tương tự: ba lớp con (Dictionary, History, Favorites) đều cài đặt cùng cặp phương thức saveToFile/loadFromFile theo cách phù hợp với cấu trúc dữ liệu nội bộ của mình. Client code có thể duyệt một mảng IPersistable* và gọi saveToFile() đồng loạt mà không cần biết kiểu cụ thể.

4.4. Hệ thống Exception

Thay vì trả về bool hoặc -1 để báo lỗi, hệ thống dùng exception. Cây kế thừa:

```

std::exception
|-- DictionaryException      (base, chứa message_)
    |-- WordNotFoundException
    |-- DuplicateWordException
    |-- FileIOException
    |-- InvalidInputException

```

Mọi exception trong hệ thống đều xuất phát từ DictionaryException, nên client có thể bắt một lần là đủ:

```

1 try {
2     dict_.addWord(w);
3     std::cout << "Da them tu.\n";
4 } catch (const DuplicateWordException& e) {
5     std::cout << "Loi: " << e.what() << "\n";
6 }

```

Ưu điểm: tách rõ luồng thành công và luồng lỗi, code chính dễ đọc hơn, không phải kiểm tra trị trả về sau mỗi lệnh.

CHƯƠNG 5

KIỂM THỬ

5.1. Kỹ thuật kiểm thử

- Phân nhóm test case theo cơ chế OOP cần kiểm tra (exception, polymorphism, encapsulation, ...).
- Test biên: chuỗi rỗng, lựa chọn ngoài phạm vi, vượt giới hạn lịch sử.
- Test tích hợp: thoát – mở lại để kiểm tra tính toàn vẹn dữ liệu.
- Test ngữ pháp: thử biên dịch các đoạn code vi phạm `const/private` để xác nhận encapsulation thực sự hoạt động.

Các tình huống kiểm thử được thực hiện trên file dữ liệu mẫu gồm 20 từ. Đặc biệt chú trọng các test case kiểm tra cơ chế OOP: exception, polymorphism, strategy.

STT	Tình huống / Cơ chế kiểm tra	Thao tác	Kết quả mong đợi	KQ
1	ExactSearch strategy hoạt động	Chọn 1, nhập hello	Hiển thị đủ nghĩa, ví dụ, đồng nghĩa	Đạt
2	Operator« của Word	Tra một từ	Output đúng format định nghĩa trong operator«	Đạt
3	Polymorphism: chuyển Strategy runtime	Chọn 1, nhập computer (sai)	ExactSearch fail → tự động chuyển sang ApproximateSearch, gợi ý computer khoảng cách 1	Đạt
4	ApproximateSearch với từ sai nặng	Nhập beautiful	Gợi ý beautiful khoảng cách 1	Đạt
5	Encapsulation: không thể truy cập trực tiếp dict_.words_	Test compile: viết code dict_.words_ trong main	Compile lỗi <i>private member</i>	Đạt
6	DuplicateWordException	Thêm từ hello đã tồn tại	Throw DuplicateWordException, message rõ ràng, chương trình không crash	Đạt

STT	Tình huống / Cơ chế kiểm tra	Thao tác	Kết quả mong đợi	KQ
7	WordNotFoundException	Sửa từ không tồn tại	Throw WordNotFoundException, được catch và in lỗi gọn	Đạt
8	Đa hình bắt lỗi qua base class	main() bắt const DictionaryException	Bắt được mọi exception con	Đạt
9	IPersistable polymorphism	Chọn lưu: gọi saveToFile() trên cả 3 đối tượng	Cả 3 file được lưu, mỗi loại theo định dạng riêng	Đạt
10	History – linked list MRU	Tra 5 từ, sau đó tra lại từ đầu tiên	Từ đó nhảy lên đầu, không bị nhân đôi	Đạt
11	History vượt giới hạn	Tra 25 từ khác nhau	Chỉ giữ 20 từ gần nhất (maxSize)	Đạt
12	Rule of Three – copy History	Tạo bản sao History h2 = h1, sửa h2, kiểm tra h1	h1 không bị ảnh hưởng (deep copy)	Đạt
13	Favorites – encapsulation kiểm soát thêm/xóa	Thêm study vào yêu thích, xóa study khỏi từ điển	study tự động biến mất khỏi yêu thích	Đạt
14	Lưu/tải – bền vững hoá	Thay đổi, thoát chương trình, mở lại	Mọi thay đổi được giữ nguyên	Đạt
15	Const correctness	Cố gọi addWord() trên const Dictionary&	Compile báo lỗi – tham chiếu const không thể gọi non-const method	Đạt
16	RAII – giải phóng tài nguyên	Thoát chương trình, kiểm tra Dr.Memory	Không rò rỉ bộ nhớ	Đạt
17	Tra chuỗi rỗng / menu sai	Nhập rỗng, chọn d	Báo lỗi gọn, không crash	Đạt

5.2. Demo

```

D:\kttt\PJ\Dictionary\TuDienA  X  +  v
Da tai 23 tu tu "data/dictionary.txt".

=====
          TU DIEN ANH-VIET (OOP)
=====
1. Tra cuu tu (chinh xac + gan dung)
2. Them tu moi
3. Sua tu
4. Xoa tu
5. Liet ke tat ca tu
6. Xem lich su tra cuu
7. Quan ly yeu thich
8. Luu du lieu xuong file
9. Tai lai du lieu tu file
0. Thoat (tu dong luu)
-----
Chon chuc nang:

=====
          TU DIEN ANH-VIET (OOP)
=====
1. Tra cuu tu (chinh xac + gan dung)
2. Them tu moi
3. Sua tu
4. Xoa tu
5. Liet ke tat ca tu
6. Xem lich su tra cuu
7. Quan ly yeu thich
8. Luu du lieu xuong file
9. Tai lai du lieu tu file
0. Thoat (tu dong luu)
-----
Chon chuc nang: 1

>>> TRA CUU TU
Nhap tu can tra: lern
Khong tim thay tu "lern" chinh xac.
Co phai ban muon tim:
  1. learn (khoang cach = 1)
  2. love (khoang cach = 3)
  3. run (khoang cach = 3)
Chon so de xem chi tiet (0 de bo qua): 1
-----
Tu tieng Anh : learn
Nghia       : học; học tập
Vi du      :
  - I learn English everyday.
Dong nghĩa  : study, acquire
-----

```

```
D:\kttf\PJ\Dictionary\TuDienA  X + v
-----
Chon chuc nang: 5

>>> DANH SACH TAT CA TU
Tong so tu: 22
 1. hello
 2. book
 3. computer
 4. run
 5. beautiful
 6. happy
 7. study
 8. mathematics
 9. algorithm
10. database
11. program
12. language
13. student
14. teacher
15. friend
16. family
17. love
18. water
19. food
20. time
21. learn
22. English

D:\kttf\PJ\Dictionary\TuDienA  X + v
=====
                TU DIEN ANH-VIET (OOP)
=====
 1. Tra cuu tu (chinh xac + gan dung)
 2. Them tu moi
 3. Sua tu
 4. Xoa tu
 5. Liet ke tat ca tu
 6. Xem lich su tra cuu
 7. Quan ly yeu thich
 8. Luu du lieu xuong file
 9. Tai lai du lieu tu file
 0. Thoat (tu dong luu)
-----
Chon chuc nang: d
Lua chon khong hop le, vui long chon lai.
```

```

D:\kttt\PJ\Dictionary\TuDienA x + v

=====
          TU DIEN ANH-VIET (OOP)
=====
1. Tra cuu tu (chinh xac + gan dung)
2. Them tu moi
3. Sua tu
4. Xoa tu
5. Liet ke tat ca tu
6. Xem lich su tra cuu
7. Quan ly yeu thich
8. Luu du lieu xuong file
9. Tai lai du lieu tu file
0. Thoat (tu dong luu)
-----
Chon chuc nang: 2

>>> THEM TU MOI
Tu tieng Anh: hello
Nghia tieng Viet (cach nhau bang '|'): Xin Chào
Vi du (cach nhau bang '|'): Hello my friend
Tu dong nghia (cach nhau bang '|'): Hi|Nihao
Loi: Tu da ton tai: "hello"

D:\kttt\PJ\Dictionary\TuDienA x + v

=====
1. Tra cuu tu (chinh xac + gan dung)
2. Them tu moi
3. Sua tu
4. Xoa tu
5. Liet ke tat ca tu
6. Xem lich su tra cuu
7. Quan ly yeu thich
8. Luu du lieu xuong file
9. Tai lai du lieu tu file
0. Thoat (tu dong luu)
-----
Chon chuc nang: 7

>>> QUAN LY YEU THICH
1. Xem danh sach yeu thich
2. Them tu vao yeu thich
3. Xoa tu khoi yeu thich
4. Xem chi tiet mot tu yeu thich
0. Quay lai menu chinh
Chon: 4
1. English
2. hello
3. run
Chon so thu tu: 2
-----
Tu tieng Anh : hello
Nghia       : xin chào; chào hỏi
Vi du       :
- Hello, how are you?
- Say hello to your mom.
Dong nghia  : hi, hey, greetings
-----

>>> QUAN LY YEU THICH
1. Xem danh sach yeu thich
2. Them tu vao yeu thich
3. Xoa tu khoi yeu thich
4. Xem chi tiet mot tu yeu thich
0. Quay lai menu chinh
Chon: |

```

CHƯƠNG 6

TỔNG KẾT CÁC KỸ THUẬT ĐÃ VẬN DỤNG

6.1. Kỹ thuật về thiết kế hướng đối tượng (OOP)

Kỹ thuật	Vận dụng tại
Encapsulation	Tất cả các lớp: trường private + getter/setter công khai. Quy ước hậu tố underscore.
Inheritance	3 cây kế thừa: IPersistable, ISearchStrategy, IUserInterface; cộng thêm hệ thống Exception.
Polymorphism	Pure virtual function trong các interface. Dictionary::search nhận const ISearchStrategy&, Application giữ IUserInterface*.
Abstraction	Các interface chỉ định nghĩa hợp đồng, ẩn cài đặt cụ thể.
Operator overloading	Word::operator==, friend operator«
Template (generic)	DynamicArray<T> dùng được cho mọi kiểu (string, Word, SearchResult, ...)
Const correctness	Mọi getter, mọi tham chiếu read-only đều là const. Đảm bảo invariant của object.
Rule of Three	History (linked list quản lý heap): copy ctor + copy assign + dtor đầy đủ.
RAII	Mọi tài nguyên cấp phát đều giải phóng trong destructor: DynamicArray, History::Node, bảng DP trong Levenshtein.
Exception handling	Thay return code bằng exception, cây kế thừa từ std::exception, polymorphism trong catch.

6.2. Các mẫu thiết kế (Design Patterns)

- **Strategy Pattern:** ISearchStrategy + ExactSearch, ApproximateSearch.

- **Facade Pattern:** Application gói toàn bộ hệ thống con.
- **Dependency Injection:** ConsoleUI và Application nhận tham chiếu Dictionary/History/Favorites thay vì tự tạo.
- **Single Responsibility Principle:** mỗi lớp một trách nhiệm. UI / data / algorithm tách 3 layer.

6.3. Kỹ thuật về thuật toán và cấu trúc dữ liệu

- Quy hoạch động: Levenshtein distance với bảng DP 2 chiều tự cấp phát.
- Quicksort tự cài cho SearchResult.
- Mảng động (template) – thay vector.
- Danh sách liên kết đơn với FIFO bounded – thay deque.
- Pruning sớm theo chênh lệch độ dài trong ApproximateSearch.

CHƯƠNG A

MÃ NGUỒN CÁC THÀNH PHẦN CHÍNH

A.1. Hàm main

```
1 // =====
2 // Tu Dien Anh-Viet (Phien ban OOP)
3 // Mon: Ky Thuat Lap Trinh
4 //
5 // File main: chi tao Application va goi run(). Toan bo logic
   duoc
6 // uy quyen cho cac doi tuong nghiep vu (Single Responsibility
   Principle).
7 // =====
8
9 #ifdef _WIN32
10 #include <windows.h>
11 #endif
12
13 #include <iostream>
14 #include "Application.h"
15 #include "Exceptions.h"
16
17 int main() {
18     #ifdef _WIN32
19         SetConsoleOutputCP(65001);
20         SetConsoleCP(65001);
21     #endif
22
23     try {
24         Application app;
25         app.initialize();
26         app.run();
27         app.shutdown();
28     } catch (const DictionaryException& e) {
29         std::cerr << "Loi he thong: " << e.what() << "\n";
30         return 1;
31     } catch (const std::exception& e) {
```

```

32         std::cerr << "Loi khong xac dinh: " << e.what() <<
           "\n";
33         return 2;
34     }
35
36     return 0;
37 }

```

A.2. Facade: Application

A.2.1. Application.h

```

1  #ifndef APPLICATION_H
2  #define APPLICATION_H
3
4  #include "Dictionary.h"
5  #include "History.h"
6  #include "Favorites.h"
7  #include "IUserInterface.h"
8
9  // =====
10 // Application - Facade Pattern
11 //
12 // Dong vai tro mat tien: gom tat ca thanh phan cua he thong
   va expose
13 // mot API don gian (initialize + run + shutdown) cho main().
14 //
15 // Quan ly vong doi cua:
16 //   - Dictionary, History, Favorites (data layer)
17 //   - IUserInterface (presentation layer)
18 //
19 // Su dung composition: app KHONG inherit cac doi tuong tren,
   ma chua chung.
20 // Sau nay neu doi UI thanh GUI, chi can sua Application, main
   giu nguyen.
21 // =====
22
23 class Application {
24 private:
25     Dictionary dict_;

```

```

26     History history_;
27     Favorites favorites_;
28     IUI* ui_; // pointer de cho phep polymorphism
29
30     static const int HISTORY_MAX_SIZE = 20;
31
32 public:
33     Application();
34     ~Application();
35
36     void initialize();
37     void run();
38     void shutdown();
39
40 private:
41     Application(const Application&);
42     Application& operator=(const Application&);
43 };
44
45 #endif // APPLICATION_H

```

A.2.2. Application.cpp

```

1  #include "Application.h"
2  #include "ConsoleUI.h"
3  #include "Exceptions.h"
4  #include <iostream>
5
6  static const std::string DICT_FILE = "data/dictionary.txt";
7  static const std::string HIST_FILE = "data/history.txt";
8  static const std::string FAV_FILE = "data/favorites.txt";
9
10 Application::Application()
11     : history_(HISTORY_MAX_SIZE), ui_(nullptr) {}
12
13 Application::~Application() {
14     delete ui_;
15 }
16
17 void Application::initialize() {
18     try {

```

```

19         dict_.loadFromFile(DICT_FILE);
20         std::cout << "Da tai " << dict_.size() << " tu tu \""
21             << DICT_FILE << "\".\n";
22     } catch (const FileIOException&) {
23         std::cout << "Khong tim thay \"" << DICT_FILE
24             << "\". Bat dau voi tu dien rong.\n";
25     }
26
27     try { history_.loadFromFile(HIST_FILE); }
28     catch (const FileIOException&) {}
29     try { favorites_.loadFromFile(FAV_FILE); }
30     catch (const FileIOException&) {}
31
32     ui_ = new ConsoleUI(dict_, history_, favorites_);
33 }
34
35 void Application::run() {
36     if (ui_ == nullptr) {
37         throw DictionaryException("Application chua duoc
38             initialize()");
39     }
40     ui_>run();
41 }
42
43 void Application::shutdown() {
44     // Cac doi tuong tu giai phong qua destructor (RAII)
45 }

```

A.3. Hệ thống Exception

```

1 #ifndef EXCEPTIONS_H
2 #define EXCEPTIONS_H
3
4 #include <exception>
5 #include <string>
6
7 // =====
8 // He thong Exception cho ung dung Tu Dien
9 //
10 // Cay ke thua:

```

```

11 //      std::exception
12 //      -- DictionaryException (base)
13 //      -- WordNotFoundException
14 //      -- DuplicateWordException
15 //      -- FileIOException
16 //      -- InvalidInputException
17 // =====
18
19 class DictionaryException : public std::exception {
20 protected:
21     std::string message_;
22 public:
23     explicit DictionaryException(const std::string& msg) :
24         message_(msg) {}
25     virtual ~DictionaryException() throw() {}
26     virtual const char* what() const throw() {
27         return message_.c_str();
28     };
29
30 class WordNotFoundException : public DictionaryException {
31 public:
32     explicit WordNotFoundException(const std::string& word)
33         : DictionaryException("Không tìm thấy từ: \"" + word +
34             "\"") {}
35 };
36
37 class DuplicateWordException : public DictionaryException {
38 public:
39     explicit DuplicateWordException(const std::string& word)
40         : DictionaryException("Từ đã tồn tại: \"" + word +
41             "\"") {}
42 };
43
44 class FileIOException : public DictionaryException {
45 public:
46     explicit FileIOException(const std::string& filename)
47         : DictionaryException("Lỗi truy cập file: \"" +
48             filename + "\"") {}
49 };

```

```

48 class InvalidInputException : public DictionaryException {
49 public:
50     explicit InvalidInputException(const std::string& msg)
51         : DictionaryException("Du lieu nhap khong hop le: " +
52                               msg) {}
53 };
54 #endif // EXCEPTIONS_H

```

A.4. Interface IPersistable

```

1  #ifndef I_PERSISTABLE_H
2  #define I_PERSISTABLE_H
3
4  #include <string>
5
6  // =====
7  // IPersistable - Interface cho cac doi tuong co the luu/tai
8  // tu file
9  //
10 // Abstract base class (pure virtual), khong the instantiate.
11 // Bat ky lop nao can persistence deu phai override 2 phuong
12 // thuc:
13 //
14 // - saveToFile(): ghi du lieu xuong file
15 // - loadFromFile(): doc du lieu tu file
16 //
17 // Hien tai duoc implement boi: Dictionary, History, Favorites.
18 // =====
19
20 class IPersistable {
21 public:
22     virtual ~IPersistable() {}
23
24     virtual void saveToFile(const std::string& filename) const
25         = 0;
26     virtual void loadFromFile(const std::string& filename) = 0;
27 };
28
29 #endif // I_PERSISTABLE_H

```

A.5. Strategy Pattern – ISearchStrategy

```

1  #ifndef I_SEARCH_STRATEGY_H
2  #define I_SEARCH_STRATEGY_H
3
4  #include <string>
5  #include "DynamicArray.h"
6  #include "Word.h"
7
8  // =====
9  // ISearchStrategy - Interface cho thuật toán tìm kiếm
   // (Strategy Pattern)
10 //
11 // Cac implementation hien co:
12 //   - ExactSearch      : tìm chính xác (case-insensitive)
13 //   - ApproximateSearch : tìm gần đúng (Levenshtein distance)
14 //
15 // Co the de dang them moi:
16 //   - PrefixSearch     : tìm theo tiền tố (Trie)
17 //   - PhoneticSearch    : tìm theo phát âm (Soundex)
18 // =====
19
20 struct SearchResult {
21     int wordIndex;
22     int score;
23
24     SearchResult() : wordIndex(-1), score(0) {}
25     SearchResult(int idx, int s) : wordIndex(idx), score(s) {}
26 };
27
28 class ISearchStrategy {
29 public:
30     virtual ~ISearchStrategy() {}
31
32     virtual void search(const DynamicArray<Word>& words,
33                        const std::string& query,
34                        DynamicArray<SearchResult>& results)
35                        const = 0;
36 };
37 #endif // I_SEARCH_STRATEGY_H

```



```

35         for (int i = 0; i < parts.size(); ++i)
36             current.addExample(parts[i]);
37     } else if (line.length() >= 7 && line.substr(0, 7)
38         == "synonym") {
39         val = line.substr(8);
40         StringUtils::split(val, '|', parts);
41         for (int i = 0; i < parts.size(); ++i)
42             current.addSynonym(parts[i]);
43     }
44 }
45
46 if (inWord && !current.getEnglish().empty()) {
47     try { addWord(current); }
48     catch (const DuplicateWordException&) {}
49 }
50 fin.close();
51 }
52
53 void Dictionary::saveToFile(const std::string& filename) const
54 {
55     std::ofstream fout(filename.c_str());
56     if (!fout.is_open()) {
57         throw FileIOException(filename);
58     }
59
60     fout << "# File du lieu tu dien Anh-Viet\n";
61     fout << "# Moi tu mot block, bat dau @english va ket thuc
62         @end\n\n";
63
64     for (int i = 0; i < words_.size(); ++i) {
65         const Word& w = words_[i];
66         fout << "@english=" << w.getEnglish() << "\n";
67         writeField(fout, "meaning", w.getMeanings());
68         writeField(fout, "example", w.getExamples());
69         writeField(fout, "synonym", w.getSynonyms());
70         fout << "@end\n\n";
71     }
72     fout.close();
73 }

```

A.7. History.cpp (Rule of Three)

```

1  History::History(int maxSize)
2      : head_(nullptr), count_(0), maxSize_(maxSize) {}
3
4  History::History(const History& other)
5      : head_(nullptr), count_(0), maxSize_(other.maxSize_) {
6      copyFrom(other);
7  }
8
9  History& History::operator=(const History& other) {
10     if (this == &other) return *this;
11     freeAll();
12     maxSize_ = other.maxSize_;
13     copyFrom(other);
14     return *this;
15 }
16
17 History::~History() {
18     freeAll();
19 }
20
21 void History::add(const std::string& word) {
22     // 1) Neu da co, xoa ban cu
23     Node* prev = nullptr;
24     Node* cur = head_;
25     while (cur) {
26         if (cur->word == word) {
27             if (prev == nullptr) head_ = cur->next;
28             else prev->next = cur->next;
29             delete cur;
30             --count_;
31             break;
32         }
33         prev = cur;
34         cur = cur->next;
35     }
36
37     // 2) Chen vao dau
38     Node* newNode = new Node(word);
39     newNode->next = head_;

```

```
40     head_ = newNode;
41     ++count_;
42
43     // 3) Cat duoi neu vuot gioi han
44     if (count_ > maxSize_) {
45         Node* p = head_;
46         while (p->next && p->next->next) p = p->next;
47         delete p->next;
48         p->next = nullptr;
49         --count_;
50     }
51 }
52
53 void History::saveToFile(const std::string& filename) const {
54     std::ofstream fout(filename.c_str());
55     if (!fout.is_open()) {
56         throw FileIOException(filename);
57     }
58     Node* cur = head_;
59     while (cur) {
60         fout << cur->word << "\n";
61         cur = cur->next;
62     }
63     fout.close();
64 }
```